Smart Camera & Video Surveillance System

A Deep Learning based Criminal Recognition and Tracking System

COMPLETE PROJECT DEEP DIVE

Every folder, every file, every line explained in simple language

Date: 04 November 2026

Final Year B.Tech (CSE) Major Project

This document is written for someone with no prior Computer Vision background. Read it top to bottom once; then use it as a reference while explaining the project to your guide or panel.

Table of Contents

1. What This Project Does (Big Picture)
2. Folder Structure - What Lives Where
3. The Environment (Python, Packages, .env)
4. End-to-End Data Flow (Frame Lifecycle)
5. File-by-File Explanation
    - 5.1 config/settings.py
    - 5.2 src/face_detector.py
    - 5.3 src/face_recognizer.py (CORE FILE)
    - 5.4 src/database.py
    - 5.5 src/email_alert.py
    - 5.6 src/detection_logger.py
    - 5.7 src/object_detector.py
    - 5.8 src/siren.py
    - 5.9 src/dashboard.py
    - 5.10 app.py
    - 5.11 templates/ (HTML pages)
    - 5.12 src/register_faces.py
    - 5.13 src/surveillance.py (terminal mode)
    - 5.14 Week-1 helper scripts
6. Core Concepts Explained Simply
7. How to Run Everything + Troubleshooting
8. Viva / Defense Questions & Answers
9. What Is Done vs What Is Remaining

1. What This Project Does (Big Picture)

This project is a smart security camera. You point a normal webcam at a door, gate, or room. The system watches the video continuously and does three jobs at the same time:

- FACE RECOGNITION - It looks at every face in the frame. If the face belongs to a person stored in its database (a "known face"), it says MATCH and shows the name and confidence. Otherwise it says Unknown.
- OBJECT DETECTION - A deep-learning model called YOLO scans the frame for common objects (person, backpack, cell phone, knife, etc.). Suspicious items are highlighted in red.
- ALERTING - When a known person is detected, the system saves a snapshot, writes the event to a database and a CSV file, sends an email with the photo attached, and plays a siren sound. It throttles alerts so you do not get 30 emails per second for the same person.

Everything is shown live in two ways: (a) a desktop OpenCV window (terminal mode), and (b) a web dashboard in the browser that streams the annotated video and shows statistics, alert history, and registered faces.

One-line summary for your report: "A real-time surveillance system that combines face recognition (face_recognition / dlib), object detection (YOLOv4-tiny via OpenCV DNN), a Flask web dashboard, and automated email + siren alerts."

2. Folder Structure - What Lives Where

Below is the complete tree. Each line has a short purpose.

```
major_project/
|-- app.py                  Flask web server (entry point for dashboard)
|-- requirements.txt        Python package list
|-- .env                    Secret email credentials (NOT committed to git)
|
|-- config/
|   \-- settings.py         All constants: paths, thresholds, flags
|
|-- src/
|   |-- face_detector.py    Haar-cascade face detection (Week 1)
|   |-- face_recognizer.py  CORE: detect + encode + match + snapshot
|   |-- database.py         SQLite (known_faces + detections tables)
|   |-- email_alert.py      SMTP email with snapshot attachment
|   |-- detection_logger.py CSV logging + stats
|   |-- object_detector.py  YOLO object detection (OpenCV DNN)
|   |-- siren.py            Generates + plays siren.wav
|   |-- dashboard.py        Background camera thread for the web app
|   |-- surveillance.py     Terminal OpenCV-window surveillance loop
|   |-- register_faces.py   Menu to add/delete/view known faces
|   |-- dataset_collector.py  Capture training images per person
|   |-- webcam_capture.py   Simple webcam preview (Week 1)
|   |-- image_face_detection.py  Detect faces in a still image
|   \-- test_recognition.py Quick test of the recognizer
|
|-- templates/              Flask HTML templates
|   |-- base.html           Common layout + CSS
|   |-- index.html          Live feed + stats
|   |-- alerts.html         Detection history table
|   \-- known_faces.html    Registered faces gallery
|
|-- models/
|   |-- haarcascade_frontalface_default.xml
|   |-- haarcascade_eye.xml
|   |-- yolov4-tiny.cfg
|   |-- yolov4-tiny.weights
|   \-- coco.names
|
|-- dataset/                PersonName/*.jpg training images
|-- known_faces/            Registered reference images
|-- snapshots/              Detection snapshots (auto-cleaned to 50)
|-- sounds/
|   \-- siren.wav           Generated automatically
|-- logs/
|   |-- faces.db            SQLite database
|   \-- detection_log.csv   Flat CSV log
\-- pdf/                    Documentation + PDF generators
```

3. The Environment (Python, Packages, .env)

Your laptop has TWO Python installations. This caused real bugs earlier, so this section matters.

Conda (base)	Python 3.12 - shown as (base) in the terminal. It does NOT have dlib/face_recognition. Do not use it for this project.
System Python	/usr/bin/python3 - Python 3.8.10. It HAS OpenCV, dlib, face_recognition, Flask, numpy, fpdf. ALWAYS use this one.

RULE: In every command run /usr/bin/python3 app.py (never plain python3).

Key packages and why each exists

- opencv-contrib-python==4.8.1.78 - video capture, drawing, DNN. Pinned to 4.8 because OpenCV 5.0 removed cv2.CascadeClassifier.
- face_recognition - wraps dlib; finds faces and produces 128-number face fingerprints (encodings).
- dlib==20.0.1 - the underlying deep-learning engine for the encoder.
- numpy==1.24.4 - arrays / math. Pinned because dlib needs this ABI.
- flask - serves the web dashboard and MJPEG video stream.
- fpdf - generates these documentation PDFs.

.env file (secrets)

Email credentials must never be hard-coded in Python. They live in a .env file in the project root:

```
SENDER_EMAIL=youraddress@gmail.com
SENDER_PASSWORD=your_16_char_app_password
ALERT_EMAIL=where_to_send_alerts@gmail.com
```

settings.py reads this file itself (function _load_env) and copies each KEY=VALUE into the operating-system environment variables. Then SENDER_EMAIL = os.environ.get("SENDER_EMAIL", ""). This means the file is not imported by any library; it is parsed manually - no python-dotenv needed.

Use a Gmail APP PASSWORD (16 characters), not your normal Gmail password. Normal passwords fail because Google blocks less-secure apps.

4. End-to-End Data Flow (Frame Lifecycle)

This is the single most important thing to understand. Follow one video frame from the camera to the browser:

```
1. cv2.VideoCapture(0) grabs a BGR frame (numpy array, H x W x 3).
2. FaceRecognizer.recognize_faces(frame):
      a. Resize frame to 25% (speed!).
      b. Convert BGR -> RGB (face_recognition needs RGB).
      c. face_locations() finds boxes via HOG/CNN model in dlib.
      d. face_encodings() turns each face into a 128-dim vector.
      e. face_distance() compares each vector to all known vectors.
      f. Smallest distance < 0.4  ->  name + confidence = 1 - distance.
      g. Scale box coordinates back x4 to original size.
3. draw_results() draws red box (match) or green box (unknown) + label.
4. ObjectDetector.detect_objects() every 3rd frame:
      a. blobFromImage() -> 416x416 normalized blob.
      b. net.forward() -> YOLO raw predictions.
      c. Filter by confidence, NMSBoxes removes duplicates.
      d. Mark suspicious classes (knife, backpack, ...).
5. draw_objects() draws blue normal boxes, red suspicious boxes.
6. Alert logic: for each MATCH, check per-name cooldown (30 s):
      save snapshot -> DB insert -> CSV log -> email -> siren.
7. Frame is stored in self.frame.
8. Browser asks /video_feed repeatedly; get_frame_jpeg() encodes
   frame to JPEG bytes and streams as multipart/x-mixed-replace.
9. Browser renders it inside <img src="/video_feed">.
```

5. File-by-File Explanation

5.1 config/settings.py - one place for all settings

Instead of scattering numbers like 0.4 or 30 across files, every module imports them from here. Change once, affects the whole system.

```
BASE_DIR = .../major_project          # absolute project root
_load_env()                           # parse .env -> os.environ

KNOWN_FACES_DIR, SNAPSHOTS_DIR, LOGS_DIR, DATASET_DIR, MODELS_DIR
FACE_CASCADE_PATH / EYE_CASCADE_PATH  # Haar XML paths
CAMERA_INDEX = 0                      # 0 = default webcam
CONFIDENCE_THRESHOLD = 0.4            # face-distance cutoff
ALERT_COOLDOWN_SECONDS = 30           # per-person alert throttle
SNAPSHOT_KEEP = 50                    # keep only newest 50 jpgs
SCALE_FACTOR = 1.3, MIN_NEIGHBORS = 5 # Haar detector tuning
FRAME_WIDTH = 640, FRAME_HEIGHT = 480
ALERT_EMAIL, SMTP_SERVER, SMTP_PORT = smtp.gmail.com, 587
SENDER_EMAIL / SENDER_PASSWORD        # from .env
YOLO_CFG / YOLO_WEIGHTS / YOLO_NAMES  # model file paths
YOLO_CONFIDENCE = 0.4, YOLO_THRESHOLD = 0.4   # confidence + NMS
SUSPICIOUS_OBJECTS = ["backpack","handbag","suitcase",
                      "cell phone","knife","scissors"]
SIREN_ENABLED = True
SIREN_SOUND_PATH = sounds/siren.wav
```

"Confidence" here means 1 minus distance, NOT a probability. A distance of 0.25 becomes 75% confidence. Lower distance = better match.

5.2 src/face_detector.py - Week 1 detection

Uses a Haar Cascade - an old but very fast classical detector. It is NOT used for recognition; it only answers "where is a face?". It is useful for the dataset collector and simple preview scripts.

```
detector = FaceDetector()
faces = detector.detect(gray_frame)     # list of (x, y, w, h)
detector.draw(frame, faces)             # green rectangles
```

How Haar works in one line: it slides small black-and-white "edge" patterns over the image and checks light/dark contrast around the eyes, nose and cheeks. It is fast on CPU but less accurate than dlib, so the main pipeline uses dlib through face_recognition instead.

5.3 src/face_recognizer.py - THE CORE FILE

This file is the heart of the project. Understand it and you can defend the whole system.

Class FaceRecognizer - what it holds:

- self.db - a FaceDatabase object (SQLite connection).
- self.known_encodings - list of 128-number numpy vectors.
- self.known_names - matching list of names (same index order).

__init__ / load_known_faces()

On startup it reads every row from the known_faces table, pulls out the stored encoding blob, converts it back to a numpy array, and keeps two parallel lists. Why parallel lists? face_distance() needs a list of vectors, and np.argmin() returns the INDEX of the closest one - that index is then used to look up the name. Simple and fast for small databases.

add_new_face(name, image_path)

Loads an image file, computes face encodings, takes the FIRST face, saves the name + encoding to the database, and appends to the in-memory lists so the change is live immediately (no restart).

add_face_from_frame(name, frame, face_location)

Same idea, but the face comes from a live camera frame. It crops the box, converts BGR->RGB, encodes, saves a reference JPEG into snapshots/, and registers it.

recognize_faces(frame) - the most asked-about method

```
small_frame = cv2.resize(frame, (0,0), fx=0.25, fy=0.25)  # 4x faster
rgb_small   = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)
face_locations  = face_recognition.face_locations(rgb_small)
face_encodings  = face_recognition.face_encodings(rgb_small, face_locations)
for encoding, location in zip(face_encodings, face_locations):
    distances = face_recognition.face_distance(self.known_encodings, encoding)
    best_match_idx = np.argmin(distances)          # closest known face
    best_distance  = distances[best_match_idx]
    if best_distance < CONFIDENCE_THRESHOLD:       # 0.4
        name = self.known_names[best_match_idx]
        confidence = 1.0 - best_distance
        is_match = True
    # coordinates were found on the 25% image -> multiply by 4
    top,right,bottom,left = location
    top*=4; right*=4; bottom*=4; left*=4
    results.append({name, confidence, is_match, location, timestamp})
```

Why resize to 25%? Face detection is the slowest step. Detecting on a small image and multiplying coordinates by 4 gives ~4x speed-up with almost no accuracy loss for typical webcam distance.

save_recognition_snapshot(frame, result, camera_id)

Creates a filename like MATCH_arun_20261104_153012.jpg (or UNKNOWN_...). Crops the face with 30 px padding so the photo is not cut at the chin, writes it to snapshots/, inserts a row into the detections table, then calls cleanup_old_snapshots() so the folder never grows forever.

cleanup_old_snapshots(keep=50)

Lists all .jpg files, sorts by modification time, and deletes everything except the newest 50. This is why your disk does not fill up during a long demo.

draw_results(frame, results)

For each face: if it is a match -> RED box + "MATCH: name (85%)". If unknown -> GREEN box + "Unknown". The colored filled rectangle behind the text keeps the label readable on any background.

5.4 src/database.py - SQLite storage

SQLite is a tiny database stored in one file (logs/faces.db). No server, no installation - perfect for a college project.

Two tables:

```
known_faces(id, name, encoding BLOB, image_path, added_date)
detections(id, name, confidence, camera_id,
           snapshot_path, timestamp, is_match)
```

The encoding is a numpy float64 array. SQLite cannot store arrays, so we call encoding.tobytes() to store raw bytes and np.frombuffer(..., dtype=np.float64) to read it back. The dtype must match exactly or the numbers become garbage.

Thread-safety (an important bug you fixed):

- Flask runs the camera in a background thread, but database calls can come from that thread AND from web requests.
- sqlite3 by default forbids using a connection from a different thread -> "SQLite objects created in a thread can only be used in that same thread".
- Fix: sqlite3.connect(path, check_same_thread=False) plus a threading.Lock() around every read/write so two threads never write at the exact same moment.

This is a great viva point: explain the race condition and why a Lock serializes access.

5.5 src/email_alert.py - email with photo attached

Builds a MIME message: a plain-text body plus the snapshot as an image attachment. Then connects to smtp.gmail.com on port 587, upgrades the connection with STARTTLS (encryption), logs in, sends, and quits.

```
server = smtplib.SMTP("smtp.gmail.com", 587)
server.starttls()                # encrypt the connection
server.login(sender_email, sender_password)
server.sendmail(sender, receiver, msg.as_string())
server.quit()
```

If SENDER_EMAIL or SENDER_PASSWORD is empty, self.enabled is False and the system skips email but keeps logging and siren. test_connection() checks login without sending a full alert.

5.6 src/detection_logger.py - CSV log + stats

Appends one row per event to logs/detection_log.csv: timestamp, name, confidence, camera_id, is_match (1/0), snapshot_path. get_stats() counts total / matches / unknown by reading the CSV. Advantages over the database: human-readable, opens in Excel, and easy to include in a report.

5.7 src/object_detector.py - YOLO object detection

YOLO = "You Only Look Once". One neural-network pass predicts all boxes and classes at the same time, which is why it is fast enough for live video. We use YOLOv4-tiny, a small version that runs on CPU.

What the code does, step by step:

- _load_network() - cv2.dnn.readNetFromDarknet(cfg, weights); set backend OPENCV and target CPU.
- _load_classes() - reads coco.names, the 80 class labels.
- detect_objects(frame):

```
blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416,416),
                             swapRB=True, crop=False)
net.setInput(blob)
outputs = net.forward(output_layers)      # raw YOLO predictions
# each detection = [center_x, center_y, w, h, objectness,
#                   class_score_0 .. class_score_79]  (all 0..1)
scores = detection[5:]
class_id = int(scores.argmax())
confidence = float(scores[class_id])
if confidence > 0.4:
    convert center/w/h -> top-left x/y
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.4, 0.4)
```

- NMS (Non-Maximum Suppression) removes duplicate boxes for the same object by keeping the highest-confidence box and deleting boxes that overlap it too much.
- draw_objects() - suspicious classes are RED with a percentage; all other classes are BLUE.
- count_suspicious() - used for the dashboard "Suspicious" counter.

Performance trick in dashboard.py: YOLO runs only every 3rd frame (frame_num % 3 == 0) while the boxes are redrawn every frame. Face recognition also runs on a 25%-size frame. This keeps the feed smooth on a CPU-only laptop.

5.8 src/siren.py - siren sound

If sounds/siren.wav is missing it is generated mathematically. It creates a sine wave whose frequency oscillates between 500 Hz and 950 Hz 1.5 times per second, applies a short fade-in/fade-out envelope (to avoid clicks), scales to 16-bit samples, and writes a mono WAV at 44100 Hz.

```
freq    = 500 + 450 * (0.5 + 0.5*sin(2*pi*1.5*t))
phase   = cumsum(2*pi*freq/sample_rate)
wave    = sin(phase)
wave   *= fade_in_and_fade_out
samples = (wave * 0.5*32767).astype(int16)
```

Playback is platform-aware: winsound on Windows, afplay on macOS, and on Linux it tries aplay, then paplay, then play. play_async() runs it in a daemon thread so the video loop never freezes while the siren plays.

5.9 src/dashboard.py - background camera thread

The web app must serve pages while the camera keeps running. Flask alone cannot do both in one loop, so a VideoProcessor thread captures and processes frames continuously and stores the latest annotated frame in self.frame.

Main loop (_capture_loop):

```
cap = cv2.VideoCapture(0); cap.set(width, 640); cap.set(height, 480)
while running:
    ret, frame = cap.read()
    faces = recognizer.recognize_faces(frame)
    frame = recognizer.draw_results(frame, faces)
    if frame_num % 3 == 0:
        self.objects = object_detector.detect_objects(frame)
    frame = object_detector.draw_objects(frame, self.objects)
    for face in faces:
        if face.is_match and cooldown passed for this name:
            snapshot = save_recognition_snapshot(...)
            email_alert.send_alert(...)
            logger.log(...)
            siren.play_async()
    draw HUD: uptime, face/object/suspicious counts
    self.faces = current_faces
    self.frame = frame
```

get_frame_jpeg() converts the latest frame to JPEG bytes with cv2.imencode(".jpg", frame). Returning bytes (not a file) lets Flask stream it directly.

last_alert_time is a dict {name: timestamp}. The 30-second check is PER PERSON, so two different people both alert immediately, but the same person does not spam. This replaced an earlier global timer that blocked alerts for everyone.

5.10 app.py - Flask web server

Routes:

- / - home page with live feed and statistics.
- /video_feed - MJPEG stream (multipart/x-mixed-replace). The browser keeps one connection open and replaces the image each time a new JPEG arrives.
- /alerts - last 100 detections from the database.
- /known_faces - registered people and their photos.
- /snapshot/<id> - serves a registered face image by id.
- /api/stats - JSON stats for programmatic use.
- /api/streaming_status - JSON: running, email on/off, siren on/off, errors.

At startup it calls processor.start(), then app.run(host="0.0.0.0", port=5000, threaded=True, use_reloader=False). use_reloader=False is critical: the auto-reloader would start a SECOND camera thread and the camera can only be opened once. The finally block stops the thread on exit.

0.0.0.0 means "listen on all network interfaces", so a phone on the same Wi-Fi can open http://YOUR_LAPTOP_IP:5000.

5.11 templates/ - HTML pages

- base.html - shared header, navigation (Home / Alerts / Known Faces), and CSS. Other pages use {% extends "base.html" %}.
- index.html - stat cards (known faces, total detections, matches, unknown, email status), the <img src="/video_feed"> live stream, and badges for currently detected faces/objects.
- alerts.html - table of recent detections with snapshot links.
- known_faces.html - gallery of registered people using /snapshot/<id>.

5.12 src/register_faces.py - manage the database

A terminal menu with five options: (1) register one image file, (2) register every image in dataset/PersonName, (3) list registered faces, (4) delete by id, (5) back. This is how you add "criminals"/known people before running surveillance.

5.13 src/surveillance.py - terminal mode

The standalone desktop version. Opens the webcam in an OpenCV window instead of a browser. Shows a HUD with known count, matches, unknown, runtime and faces/sec. Keys: q quit, s save snapshot, a add face live, l show logs, r refresh known faces. On exit it prints a session summary. Use this for a simple offline demo; use app.py for the web demo.

5.14 Week-1 helper scripts

- webcam_capture.py - minimal webcam preview to check the camera works.
- image_face_detection.py - detects faces in a single still image.
- dataset_collector.py - captures N photos of a person into dataset/Name/ for building the dataset.
- test_recognition.py - loads the recognizer and tests matching quickly.

6. Core Concepts Explained Simply

What is a face encoding?

Think of a fingerprint made of 128 numbers. The dlib model looks at a face and outputs 128 measurements (distances between eyes, nose width, jaw shape, etc.). Two photos of the same person produce very similar 128-number vectors; different people produce far-apart vectors.

How is a match decided?

Compute the Euclidean distance between the new vector and every known vector. Distance 0 = identical. If the smallest distance is below 0.4 we call it a match. Confidence = 1 - distance. Typical same-person distance is 0.3-0.4; different people are usually above 0.5-0.6.

If matches are wrong, lower CONFIDENCE_THRESHOLD (stricter). If the system misses the right person, raise it (looser).

BGR vs RGB

OpenCV loads images as Blue-Green-Red; face_recognition/dlib expect Red-Green-Blue. Forgetting cv2.cvtColor(..., COLOR_BGR2RGB) gives wrong colors and bad encodings. That is why every recognition path converts.

Haar Cascade vs YOLO vs dlib

- Haar Cascade - classical, very fast, detects faces only, less accurate.
- dlib / face_recognition - deep model, accurate face detection + the 128-D encoder. Slower, so we run it on a resized frame.
- YOLO - deep model for 80 object classes, one pass, good speed/accuracy balance on CPU in tiny form.

MJPEG streaming

MJPEG is just a sequence of JPEG images. Flask sends a special HTTP response with boundary "frame"; the browser replaces the displayed image each time a new JPEG arrives. No WebRTC, no plugins - simple and reliable for a LAN demo.

Threads and why the Lock matters

A thread is a lightweight worker inside the same program. The camera loop and Flask request handlers are different threads sharing one SQLite connection. The Lock ensures only one thread reads/writes the database at a time, preventing corruption and "database is locked" errors.

7. How to Run Everything + Troubleshooting

Register faces

```
/usr/bin/python3 src/register_faces.py
```

Terminal surveillance (OpenCV window)

```
/usr/bin/python3 src/surveillance.py
```

Web dashboard

```
/usr/bin/python3 app.py
# then open http://localhost:5000
```

Generate PDFs

```
/usr/bin/python3 pdf/generate_deep_dive.py
```

Common problems and fixes

- ModuleNotFoundError: No module named dlib -> you used conda python. Run /usr/bin/python3 instead.
- cv2 has no attribute CascadeClassifier -> OpenCV 5 installed. Install opencv-contrib-python==4.8.1.78.
- Email not sending -> use a Gmail App Password; check .env values; port must be 587 with starttls.
- Camera already in use -> another python process or the OpenCV window is open. Close it; only one process can open camera 0.
- Webcam shows black -> try CAMERA_INDEX=1 in settings.py; check OS camera permissions.
- Slow feed -> detection every 3rd frame, 25% resize; close other apps; use YOLOv4-tiny (already default).
- Unknown is shown for a registered person -> poor lighting or face angle; re-register with a clear front-facing photo; optionally raise the threshold slightly.

8. Viva / Defense Questions & Answers

Q1. Why face_recognition instead of Haar Cascade for recognition?

A: Haar only detects where a face is. Recognition needs a numeric identity signature; dlib/face_recognition gives a 128-D encoding that can be compared by distance.

Q2. What are the 128 numbers?

A: Learned facial measurements from a deep network. Same person -> similar vector; different people -> distant vectors.

Q3. Why is the threshold 0.4?

A: Empirically, same-person distances cluster around 0.3-0.4. 0.4 balances false accepts vs false rejects. It is configurable in settings.py.

Q4. How do you speed up real-time performance?

A: Detect on a 25% frame, run YOLO only every 3rd frame, use YOLOv4-tiny on CPU, and draw cached boxes every frame.

Q5. How do you prevent email spam?

A: A per-person cooldown dict: after alerting for a name, wait 30 seconds before alerting again for that name.

Q6. How is the database made thread-safe?

A: check_same_thread=False plus a threading.Lock around every operation, because Flask and the camera run in different threads.

Q7. Why SQLite and not MySQL?

A: Zero setup, single file, enough for a single-camera prototype. It is ACID-compliant and easy to back up.

Q8. How does live video reach the browser?

A: MJPEG: Flask streams successive JPEG frames as multipart/x-mixed-replace; the <img> tag updates automatically.

Q9. What happens on an unknown face?

A: It is drawn green as Unknown, counted, and it does NOT trigger email or siren (only known matches alert).

Q10. What are the limitations / future work?

A: Single camera, no tracking across frames, no anti-spoofing (photo attack), CPU-only. Future: multi-camera, DeepSORT tracking, liveness detection, GPU acceleration.

Q11. Why is use_reloader=False important?

A: The Flask reloader launches two processes; each would open the camera and upload the model, causing "camera busy" and double alerts.

Q12. How is the siren generated?

A: A frequency-modulated sine wave (500-950 Hz) written to WAV, played in a background thread so video never freezes.

9. What Is Done vs What Is Remaining

Completed and tested

- Face detection (Haar) and dataset collection - Week 1.
- Face recognition with dlib 128-D encodings and SQLite storage.
- Terminal surveillance with live boxes, snapshots, logs.
- YOLO object detection with suspicious-object highlighting.
- Flask web dashboard: live MJPEG, stats, alerts, known faces.
- Email alerts with snapshot attachment.
- Siren alert generated and played asynchronously.
- Per-person 30 s alert cooldown and snapshot auto-cleanup.
- All PDF documentation generated.

Remaining (future scope)

- Multi-camera support (CAM_01, CAM_02 ...) with per-camera threads.
- Behavior analysis (loitering, intrusion zone, running).
- Anti-spoofing / liveness detection to stop photo attacks.
- Object tracking (DeepSORT) to follow a person across frames.
- GPU acceleration and model quantization for higher FPS.

Current status: approximately 80% complete. Core pipeline, dashboard, YOLO, email, and siren are all implemented and demonstrably working.